

OPF – Optimum Path Forest (*Floresta de Caminhos Ótimos*)

André Tavares da Silva

andre.silva@udesc.br

Objetivos

- O que é OPF
- Definindo adjacências
- Definindo protótipos
- Definindo função de custo
- Teste do classificador
- Exemplo

OPF

- OPF não é um único classificador, mas um *framework* para criação de classificadores baseados em caminho de florestas ótimos
 - pode ser supervisionado, não supervisionado ou semi-supervisionado
 - grafo Knn / grafo completo

Motivação

- Tentativa de aliar eficiência no treinamento e eficácia na classificação;
- Não necessitar de muitos exemplos para treinamento (diferente de muitos classificadores);
- Baseado na IFT (imagem como grafo para segmentação de pixels) visando a “segmentação” de características em classes;

Treinamento da OPF

- Ao extrair características de uma imagem ou utilizar uma base de dados qualquer, os dados das amostras (vetores de características) são nós do grafo;
- O objetivo é fazer que amostras de uma classe fiquem em uma partição do grafo (floresta) e das outras classes em outras partições.
- A diferença da OPF para outros classificadores semelhantes é a eurística que leva a fazer essa partição.

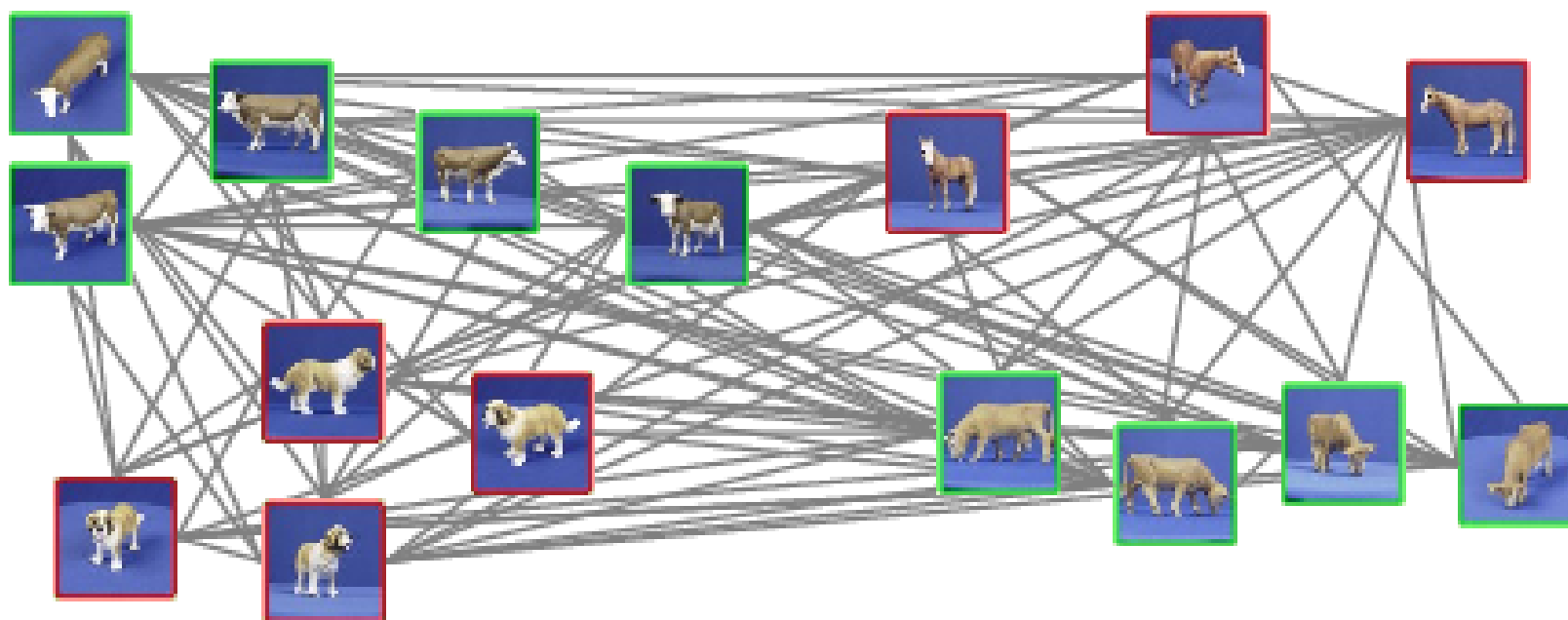
Treinamento

- Vetores de características são os nós de um grafo;
- As arestas (adjacências) podem ser representadas pela distância (similaridade) entre os vetores de característica;
- Como definir a adjacência?

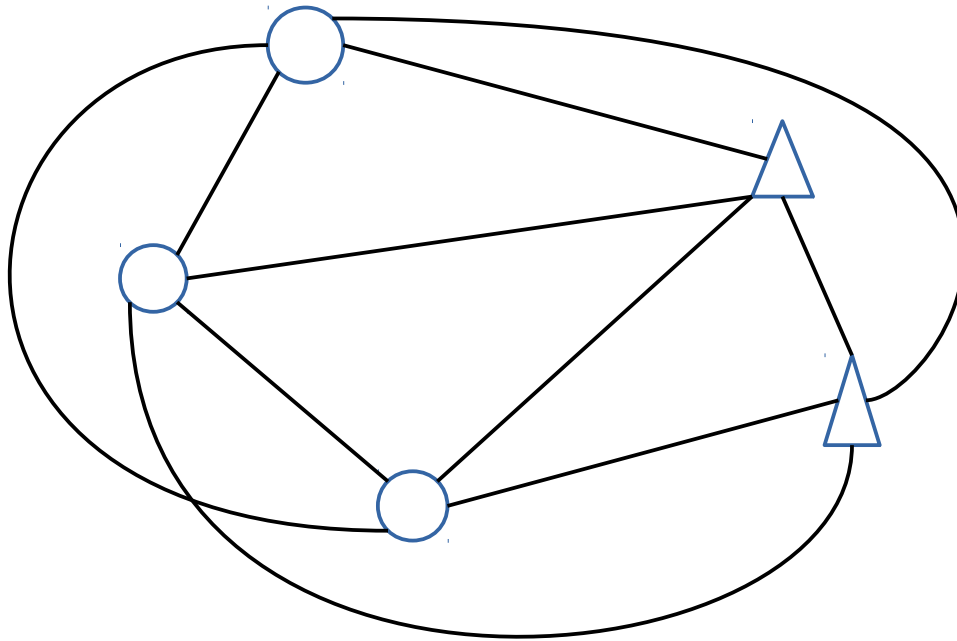
Treinamento

- Como definir a adjacência?
 - Podemos fazer um grafo Knn onde cada amostra é ligada aos seus k vizinhos mais próximos;
 - Podemos ligar todos contra todos (grafo completo);
 - Geralmente o grafo completo é mais eficiente em termos de precisão/acurácia mas tem um custo computacional maior.

Grafo Completo



Grafo Completo



Treinamento

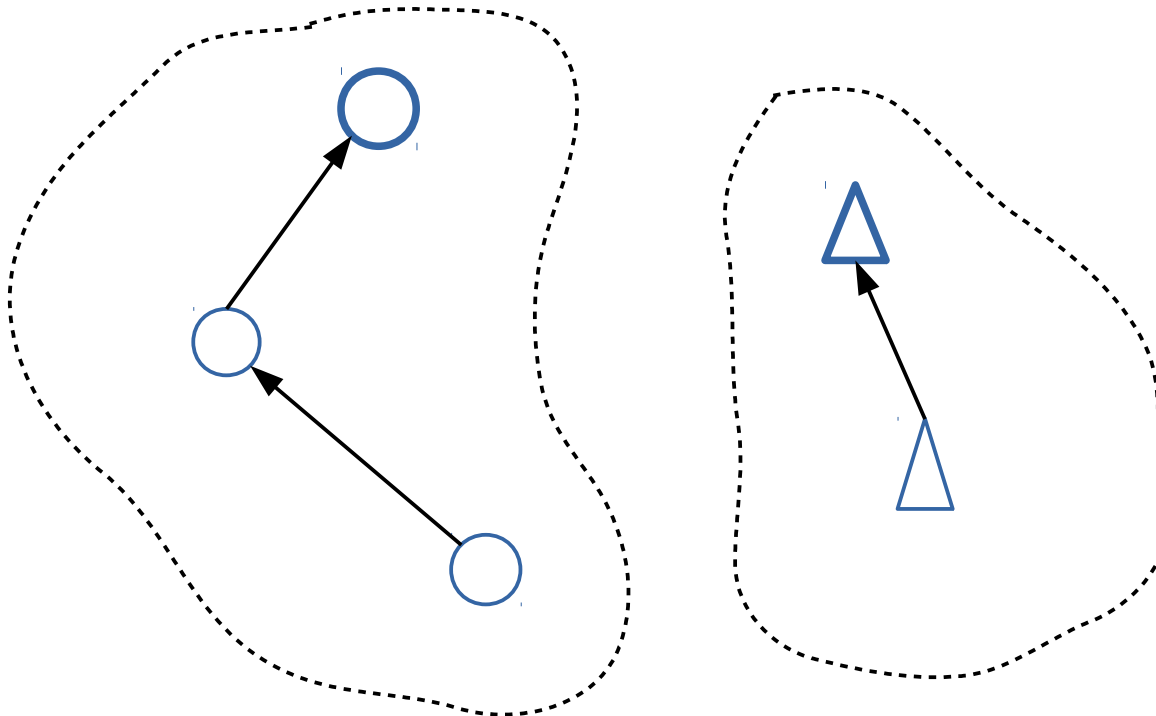
- Definida a adjacência, como determinar o processo de conquista entre as amostras?
- Como particionar o grafo de forma que tenhamos em cada agrupamento apenas amostras da mesma classe?
- Como o objetivo é segmentar o espaço de características, são escolhidos alguns nós sementes que irão “conquistar” outros nós competindo entre si. Assim como algoritmos de crescimento de regiões ou *watershed*.

Treinamento

- As amostras conquistadas recebem o mesmo rótulo das sementes (protótipos);
- Como determinar quem são os nós semente?
 - Na OPF esses nós semente são chamados de **protótipos** e são semelhantes aos vetores de suporte da SVM;
 - Os protótipos devem ser elementos representativos de cada classe;
 - Temos pelo menos um protótipo por classe, podendo ter mais de um por classe dependendo do problema.

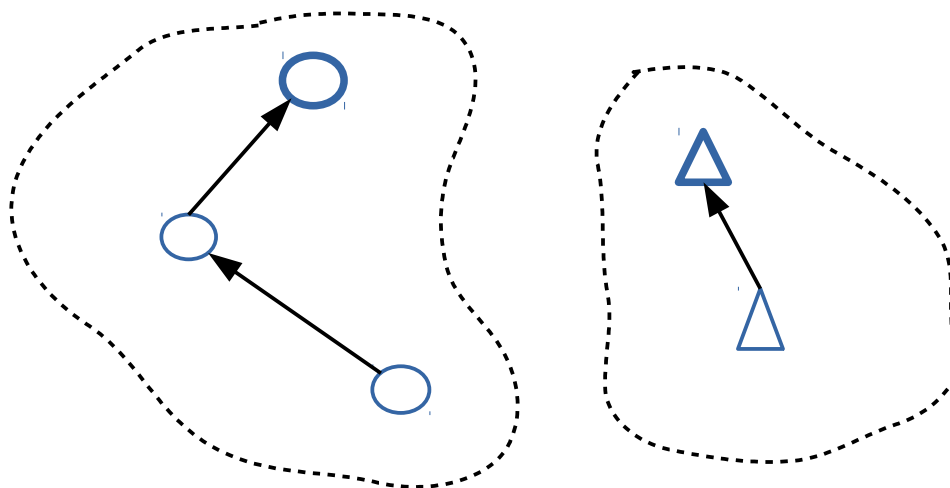
Treinamento

- Quando os protótipos conquistam amostras, a partição do espaço não é explícita como na maioria dos classificadores lineares ou não.



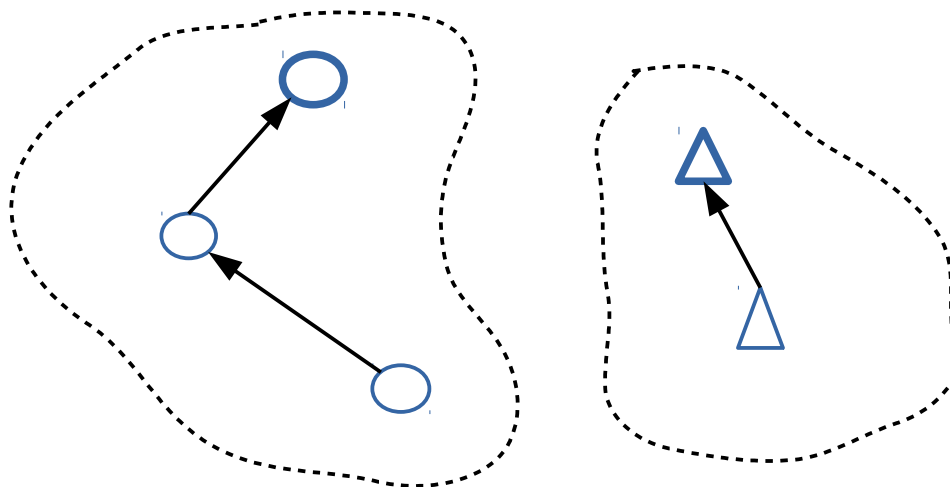
Treinamento

- A região de interesse de cada protótipo é definida por uma **árvore de caminhos ótimos**, onde a raiz é o protótipo e os nós estão mais fortemente conexos àquele protótipo do que qualquer outro elemento de qualquer outra árvore.



Treinamento

- A região de interesse de cada protótipo é definida por uma **árvore de caminhos ótimos**, onde a raiz é o protótipo e os nós estão mais fortemente conexos àquele protótipo do que qualquer outro elemento de qualquer outra árvore.



Cada floresta será uma OPT e o conjunto de OPTs é uma floresta $\rightarrow$ OPF

Treinamento

- Como determinar que um nó é mais conexo a uma árvore ou outra? Como definir o **melhor** caminho?
- Através de uma **função de custo**, que precisa ser suave (monotonicamente crescente,...).
- OPF pode ser considerado uma generalização de Dijkstra, pois a função de custo não precisa ser necessariamente $fsum$, embora precise ser uma função suave.

Treinamento

- Os protótipos, que representam as classes, iniciam o processo competindo entre si para conquistar os outros elementos do conjunto de treinamento;
- Quando uma amostra conquista outra amostra, a amostra conquistada recebe o rótulo da amostra que a conquistou;
- Esse processo de conquista, vai desenhando a fronteira de decisão (define a região de interesse de cada árvore).

Treinamento

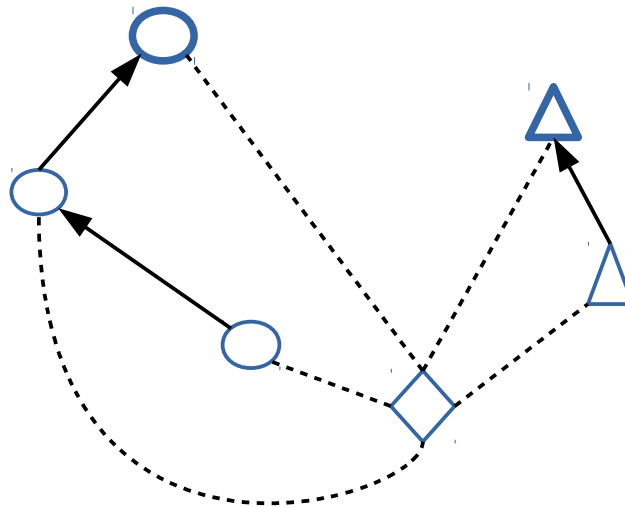
- A fase de treinamento do classificador OPF, portanto, é realizada da seguinte forma:
 - Obtendo como entrada um grafo;
 - Através do grafo, encontrar os protótipos;
 - O classificador, através de um processo iterativo cria as árvores;
 - A saída do classificador é a floresta de caminhos ótimos.

Treinamento

- Para definir um classificador OPF é preciso determinar:
 1. Qual tipo de **adjacência** do grafo (completo, Knn,...);
 2. Como escolher os **protótipos**
 3. Qual a **função de custo** (fmax, minmax,...)

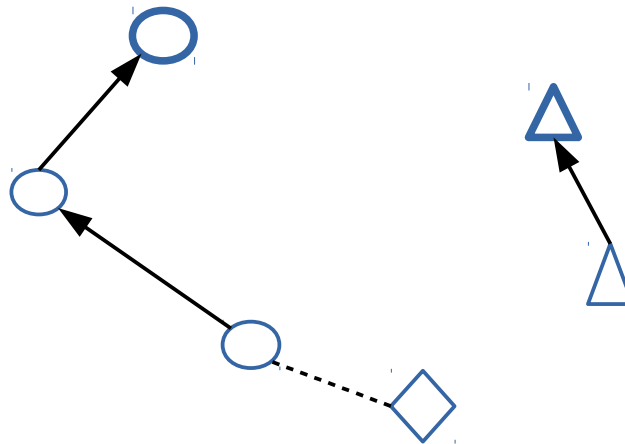
Teste

- A fase de teste consiste em, dado um exemplo do conjunto de teste, ligar ele a todos os nós da OPF e verificar qual nó conquistará esta amostra.



Teste

- A fase de teste consiste em, dado um exemplo do conjunto de teste, ligar ele a todos os nós da OPF e verificar qual nó conquistará esta amostra.

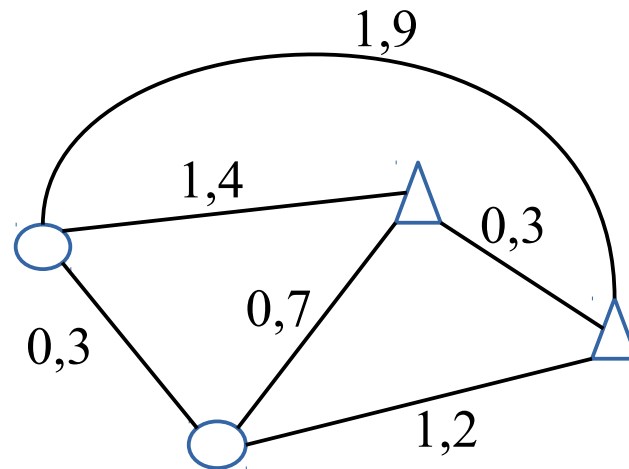


Exemplo

- São calculadas as similaridades entre as amostras do conjunto de treinamento para formar o grafo.
- Estas similaridades podem ser calculadas por qualquer métrica, como por exemplo, distância euclidiana e Manhattan. Também podemos ter outras formas de calcular essa “distância”. Para impressão digital, podemos ter diferentes minúcias para cada exemplo, assim como algoritmos como SIFT/SURF geram diferentes quantidades de elementos (descritores). Neste caso, a função de distância depende da aplicação.

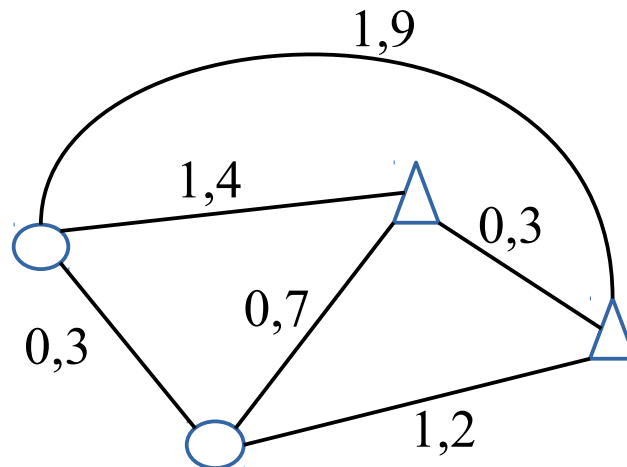
Exemplo

- Na OPF não precisamos definir *kernel* ou outra função. Basta definir a distância entre amostras:



Exemplo

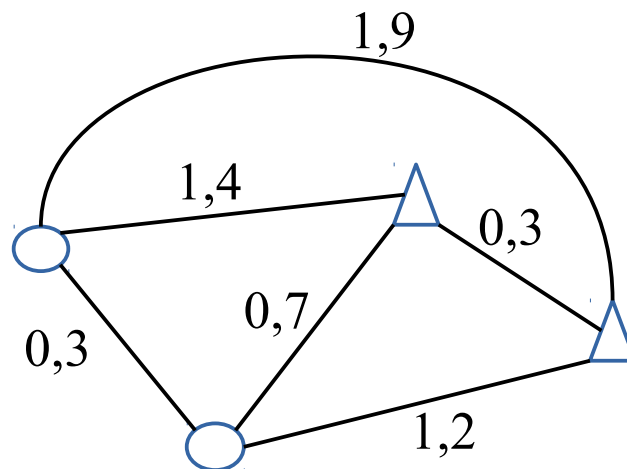
- Na OPF não precisamos definir *kernel* ou outra função. Basta definir a distância entre amostras:



1. definida a adjacência: grafo completo.

Exemplo

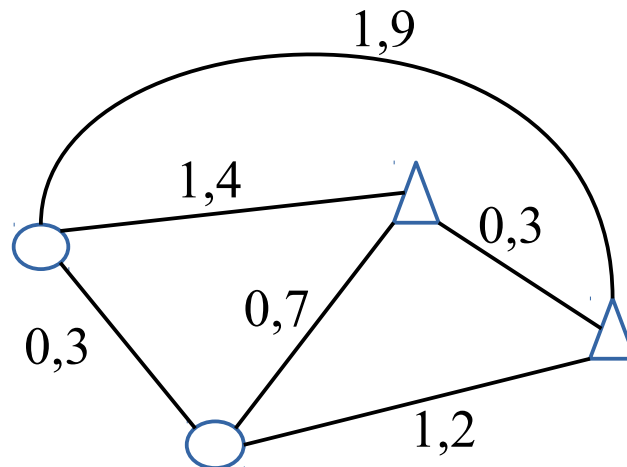
- Na OPF não precisamos definir *kernel* ou outra função. Basta definir a distância entre amostras:



1. definida a adjacência: grafo completo.
2. como escolher os protótipos?

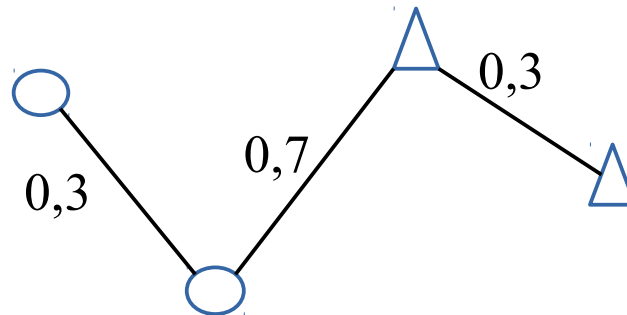
Exemplo

- Para escolher os protótipos, o melhor é escolher os nós mais propensos a dar erro, ou seja, aqueles nós mais próximos a elementos da outra classe.



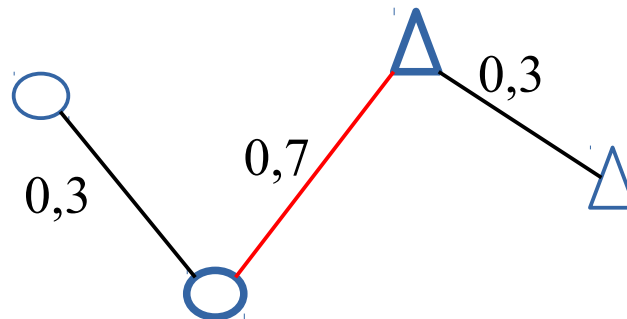
Exemplo

- Uma forma de fazer isso é calcular a MST (*minimum spanning tree*) no grafo.



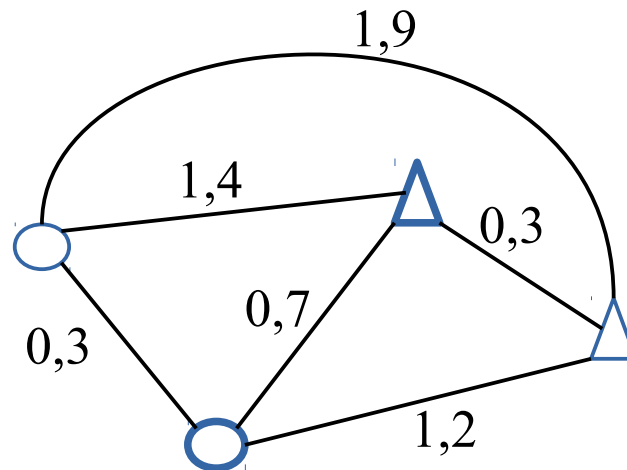
Exemplo

- Uma forma de fazer isso é calcular a MST (*minimum spanning tree*) no grafo.
- Calculada a MST, os elementos de classes diferentes ligados por uma aresta da MST são escolhidos como protótipos.



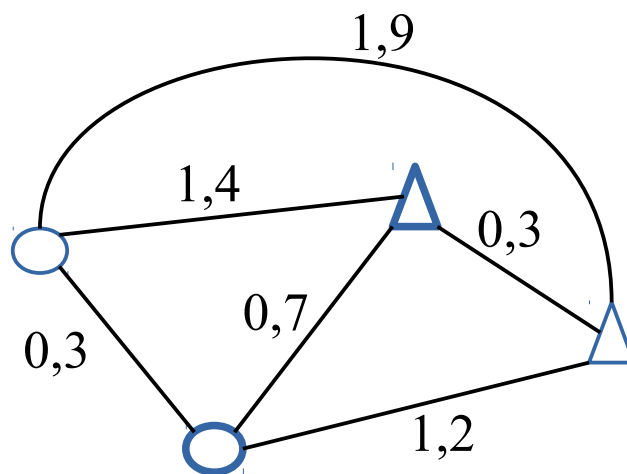
Exemplo

- Definidos os protótipos, eles são marcados no grafo completo.



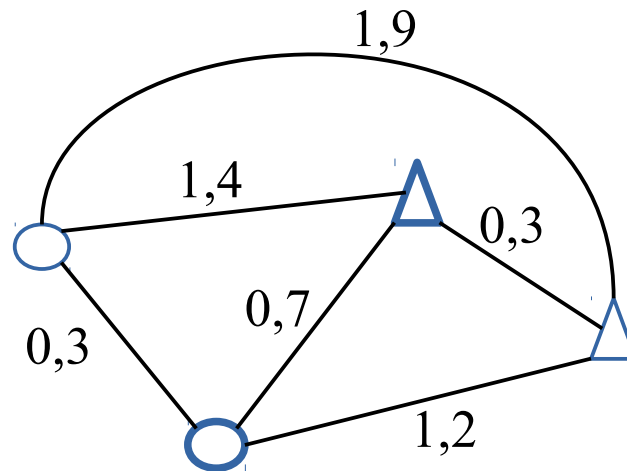
Exemplo

- Para iniciar o processo de competição entre as amostras, é atribuído um custo para cada nó. Nosso problema é de otimização (menor caminho).



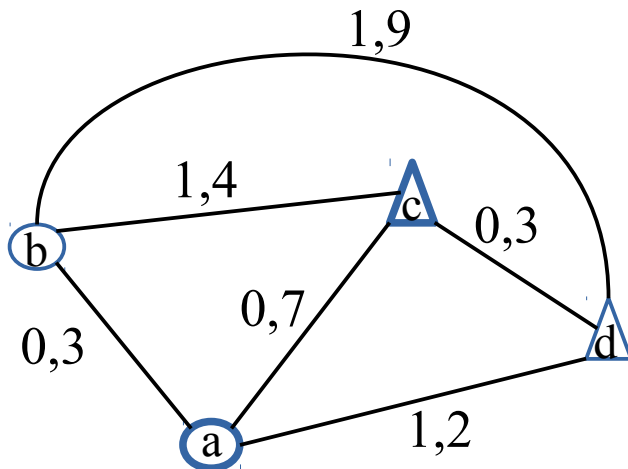
Exemplo

- Para iniciar o processo de competição entre as amostras, é atribuído um custo para cada nó. Nosso problema é de otimização (menor caminho).
- É criada então uma fila de prioridades na qual os protótipos recebem custo zero e os demais recebem custo infinito.



Exemplo

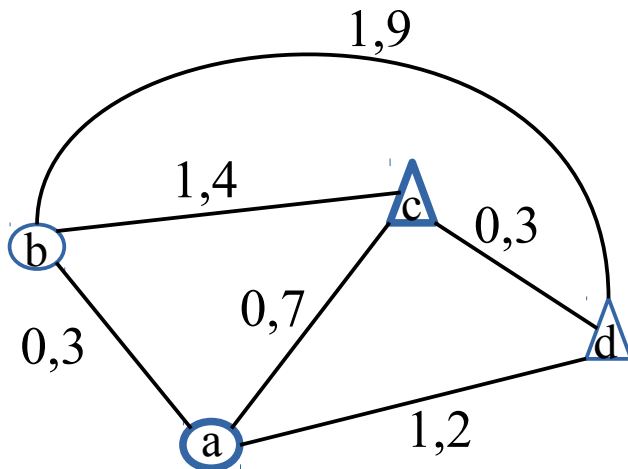
- Para iniciar o processo de competição entre as amostras, é atribuído um custo para cada nó. Nosso problema é de otimização (menor caminho).
- É criada então uma fila de prioridades na qual os protótipos recebem custo zero e os demais recebem custo infinito.



a	b	c	d
0	∞	0	∞

Exemplo

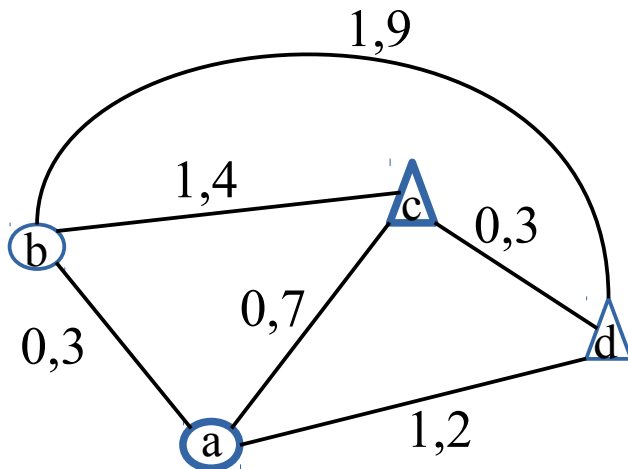
- Precisamos definir qual a **função de custo** a ser utilizada para definir o melhor caminho de um nó para a árvore (ou sua raiz). Podemos utilizar qualquer função suave (monotonicamente crescente,...).
- Dijkstra utiliza *fsum* para calcular o menor caminho de um nó a outro do grafo, por exemplo.



a	b	c	d
0	∞	0	∞

Exemplo

- Neste exemplo vamos utilizar a função $fmax$, que é a função padrão da OPF. É o valor da maior aresta ao longo do caminho até o protótipo.
- O objetivo disso é evitar problemas do tipo cadeia. No artigo da OPF são explicados os motivos do uso dessa função.



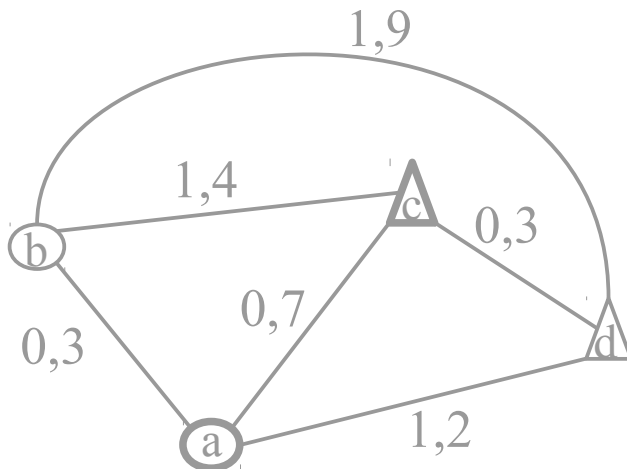
a	b	c	d
0	∞	0	∞

Relembrando...

1. Adjacência: **grafo completo**
2. Escolha dos protótipos: **MST**
3. Função de Custo: ***f_{max}***

Exemplo

- Para construir a floresta, retiramos o primeiro elemento da fila. O melhor caminho até “a” (custo zero) já foi atingido.
- Quando um elemento sai da fila, o custo dela não pode ser melhorado e está elegível a conquistar outras amostras.

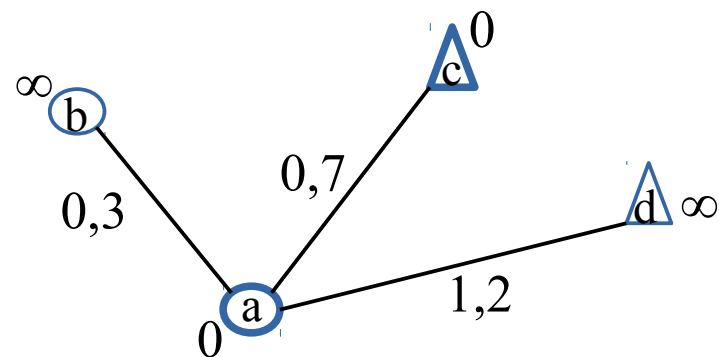
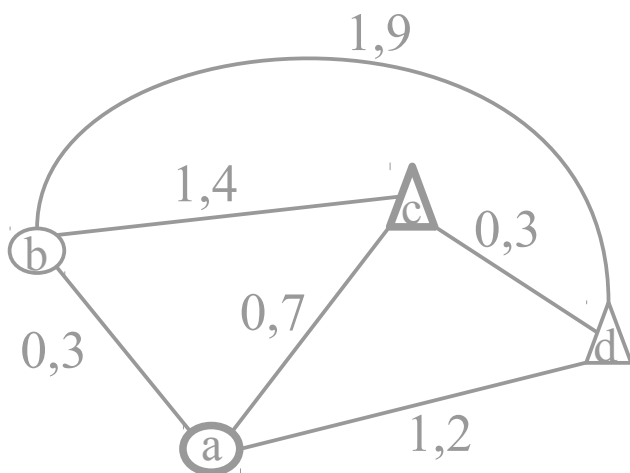


a

a	b	c	d
0	∞	0	∞

Exemplo

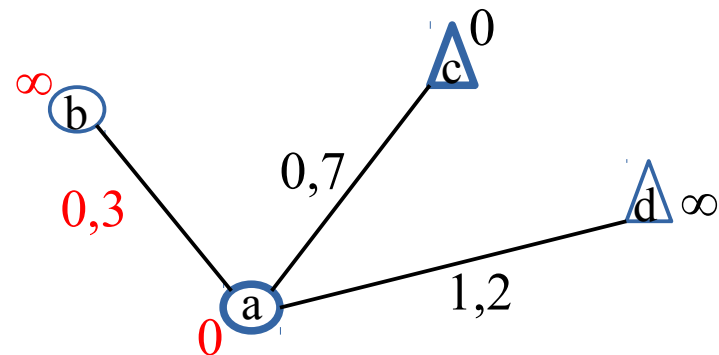
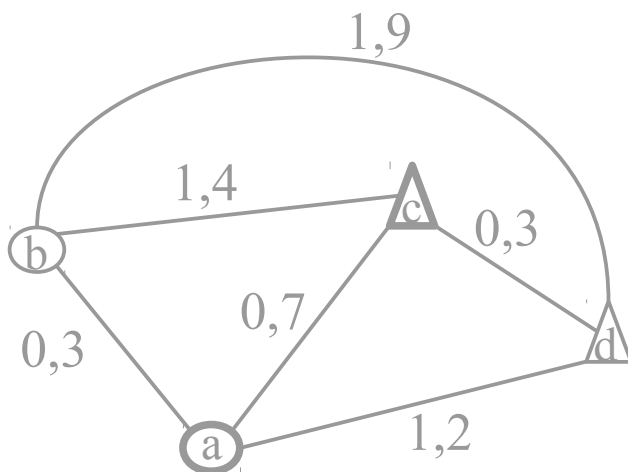
- O nó “a” vai tentar conquistar todas as amostras.



a	b	c	d
0	∞	0	∞

Exemplo

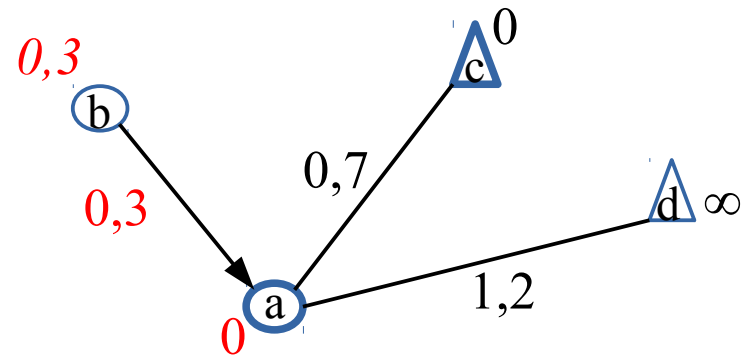
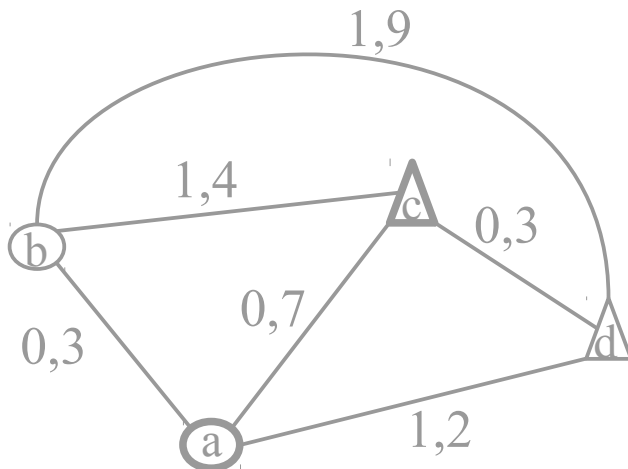
- O nó “a” vai tentar conquistar todas as amostras.
- O custo do nó “b” é infinito. O custo entre “a” e “b” é máximo entre o custo de “a” (zero) e a distância entre “a” e “b” (0,3). Como 0,3 é menor que infinito, “a” conquista “b”.



a	b	c	d
0	∞	0	∞

Exemplo

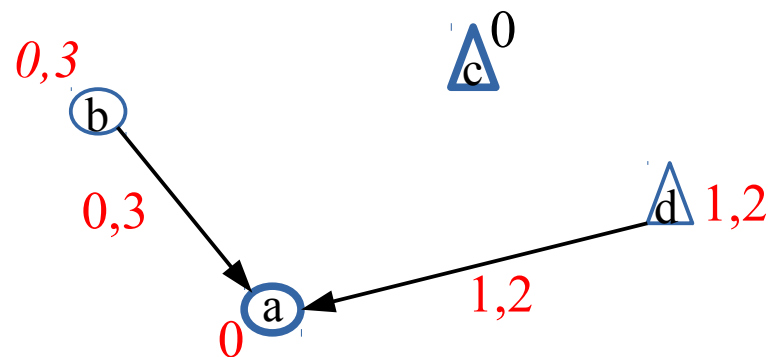
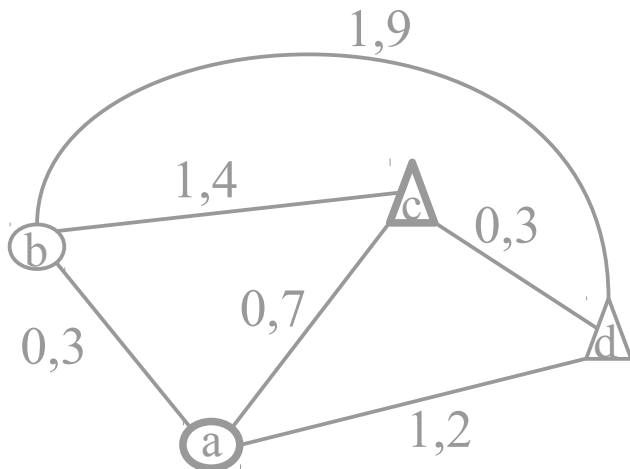
- O custo do nó “b” é infinito. O custo entre “a” e “b” é máximo entre o custo de “a” (zero) e a distância entre “a” e “b” (0,3). Como 0,3 é menor que infinito, “a” conquista “b”.
- Agora temos um mapa de predecessores de “b” para “a”.



a	b	c	d
0	0,3	0	∞

Exemplo

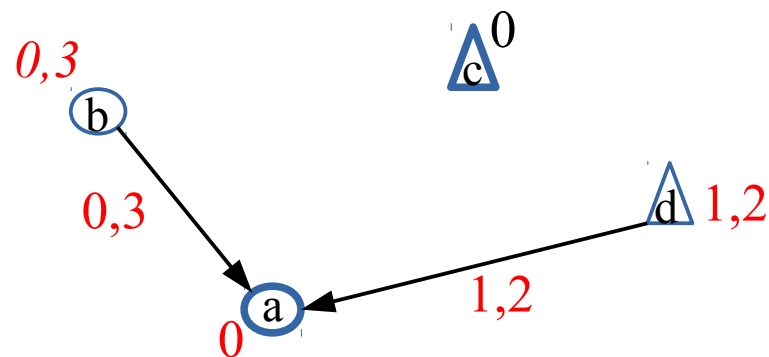
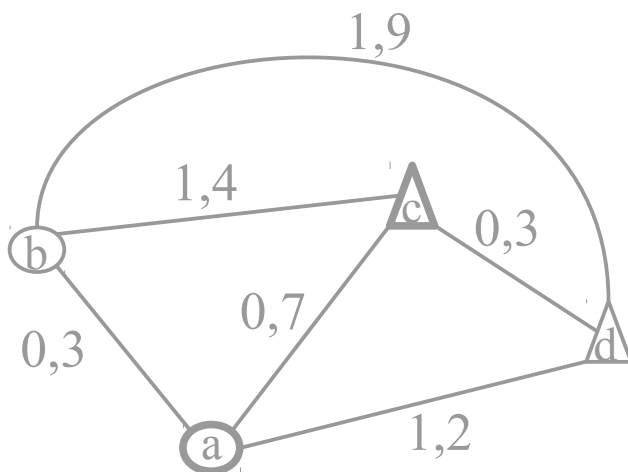
- O mesmo ocorre de “d” para “a”.
- Inicialmente conquista todos os elementos menos os protótipos, pois os protótipos tem custo zero.



a	b	c	d
0	0,3	0	1,2

Exemplo

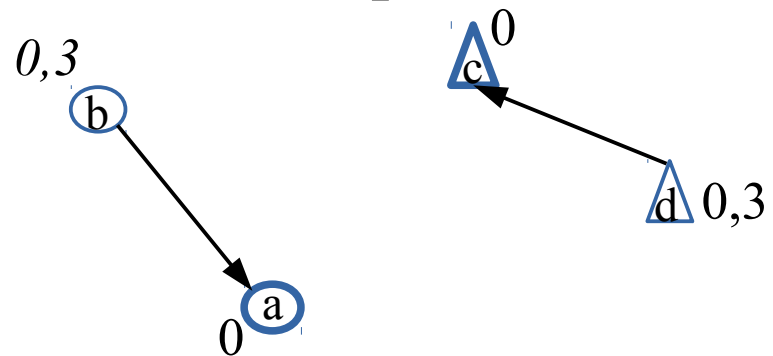
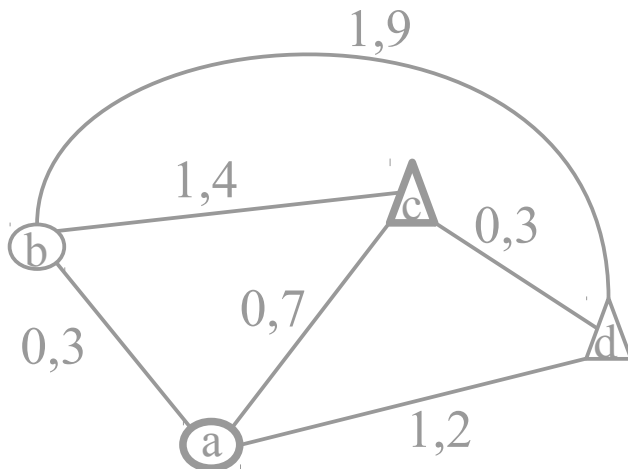
- O próximo a sair da fila é o de menor custo (nó “c”).
- Ele vai tentar conquistar os outros. Não consegue conquistar o “a” que já é protótipo.



a	b	c	d
0	0,3	0	1,2

Exemplo

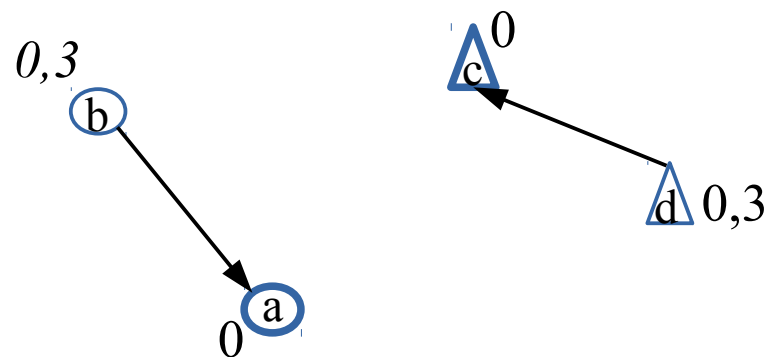
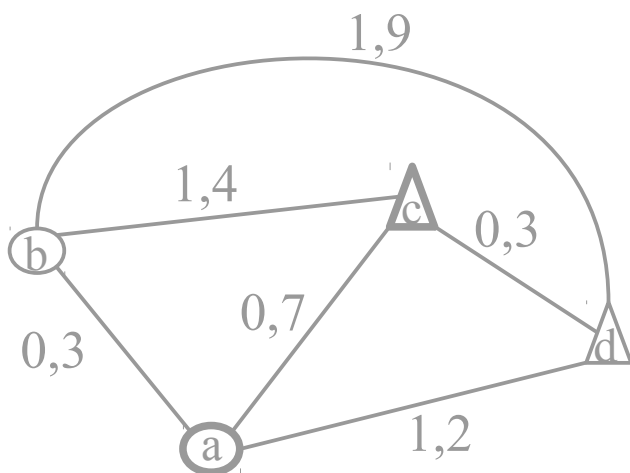
- O custo entre “b” e “c” é o máximo entre zero e 1,4. Como o custo de “b” é 0,3 o nó “c” não consegue conquistar ele.
- O custo entre “d” e “c” é o máximo entre zero e 0,3. Como o custo de “d” é 1,2 o nó “c” conquista ele.



a	b	c	d
0	0,3	0	0,3

Exemplo

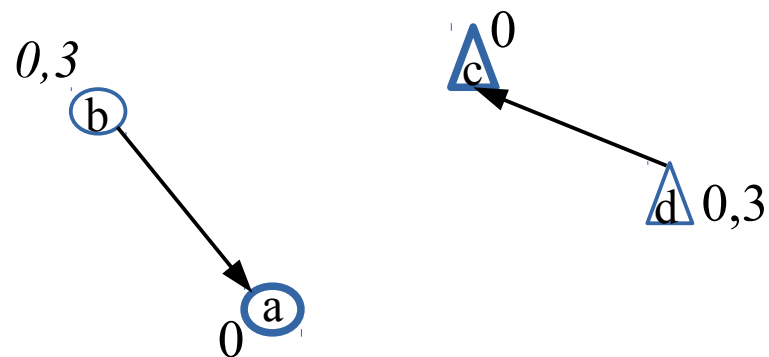
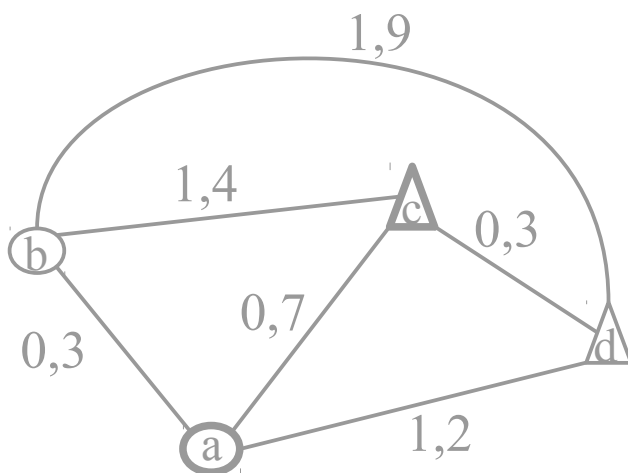
- O próximo a sair da fila é o “b”, que tenta conquistar os outros mas não consegue. Os protótipos “a” e “c” tem custo zero e “d” tem custo 0,3 (custo de “b” a “d” é 1,9).



a	b	c	d
0	0,3	0	0,3

Exemplo

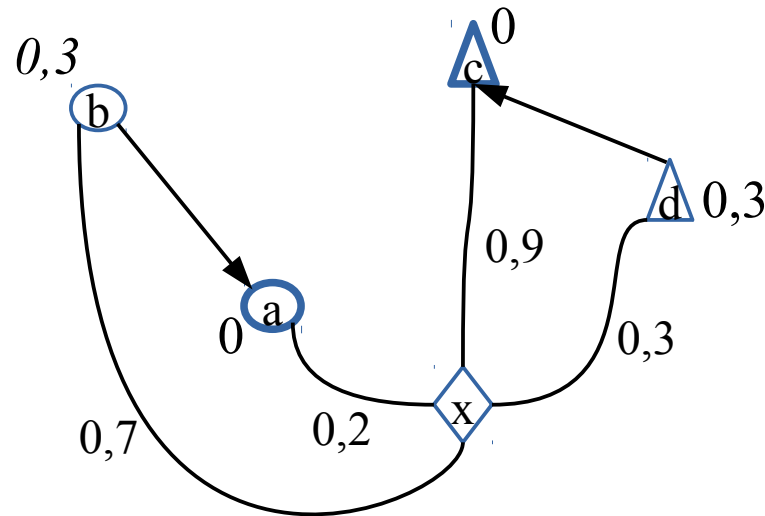
- O próximo a sair da fila é o “b”, que tenta conquistar os outros mas não consegue. Os protótipos “a” e “c” tem custo zero e “d” tem custo 0,3 (custo de “b” a “d” é 1,9).
- “d” por sua vez também não conquista ninguém.



a	b	c	d
0	0,3	0	0,3

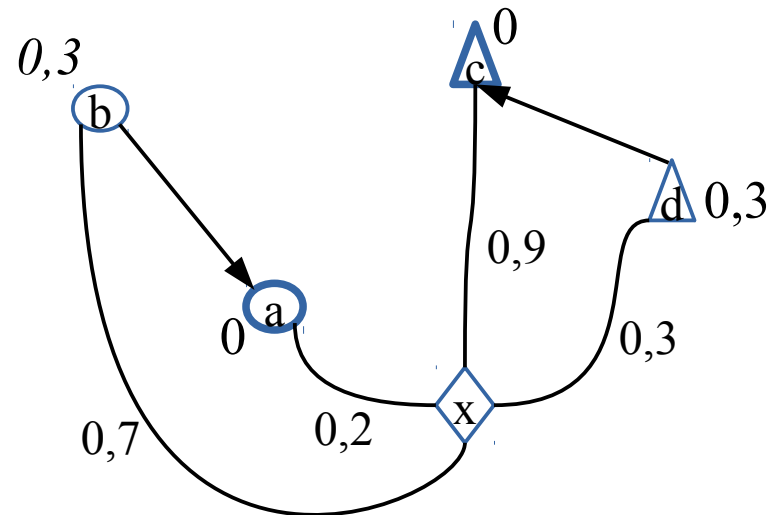
Fase de teste

- Pega-se cada amostra do conjunto de teste e calcula a distância a cada amostra do conjunto da floresta.



Fase de teste

- Verificar qual o menor custo entre as amostras do conjunto de teste (*minmax*).



Custo de “x” a “a”: máximo entre 0 e 0,2 = **0,2**

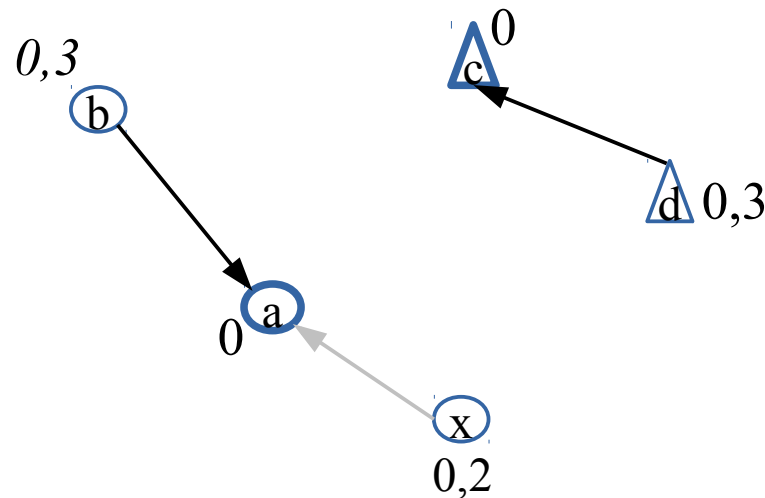
Custo de “x” a “b”: máximo entre 0,3 e 0,7 = 0,7

Custo de “x” a “c”: máximo entre 0 e 0,9 = 0,9

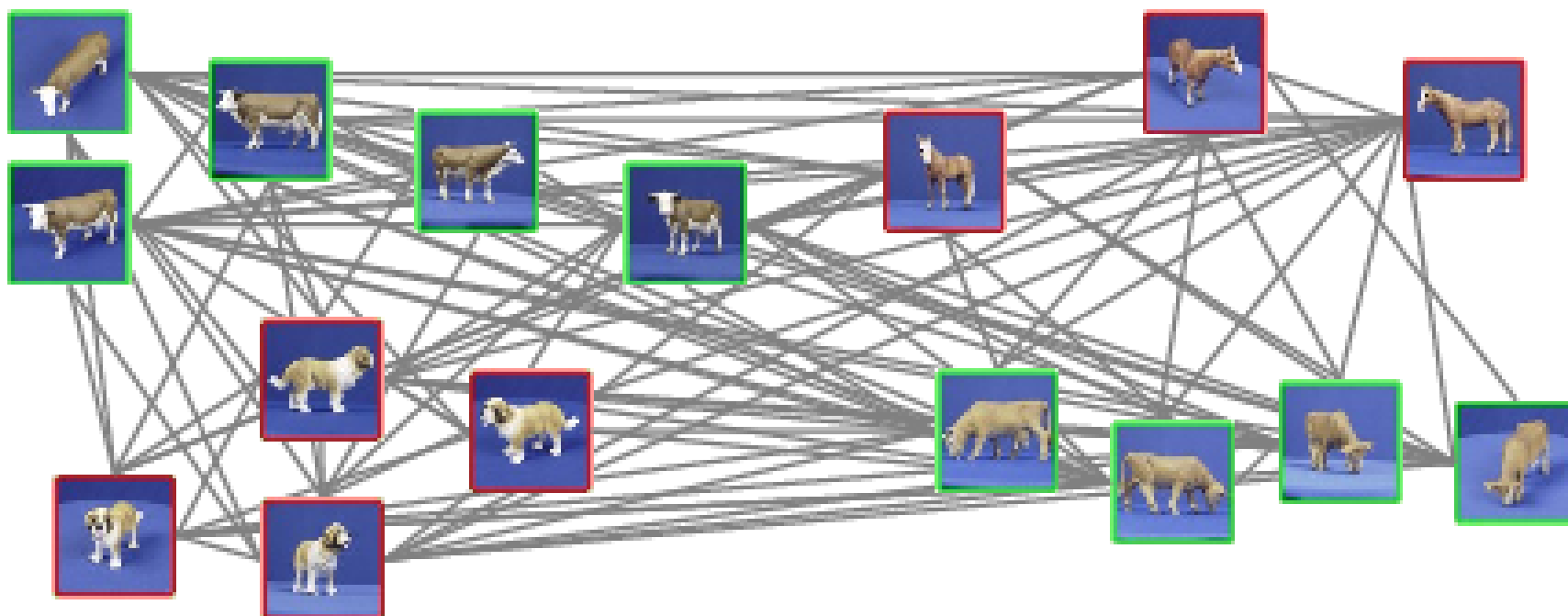
Custo de “x” a “d”: máximo entre 0,3 e 0,3 = 0,2

Fase de teste

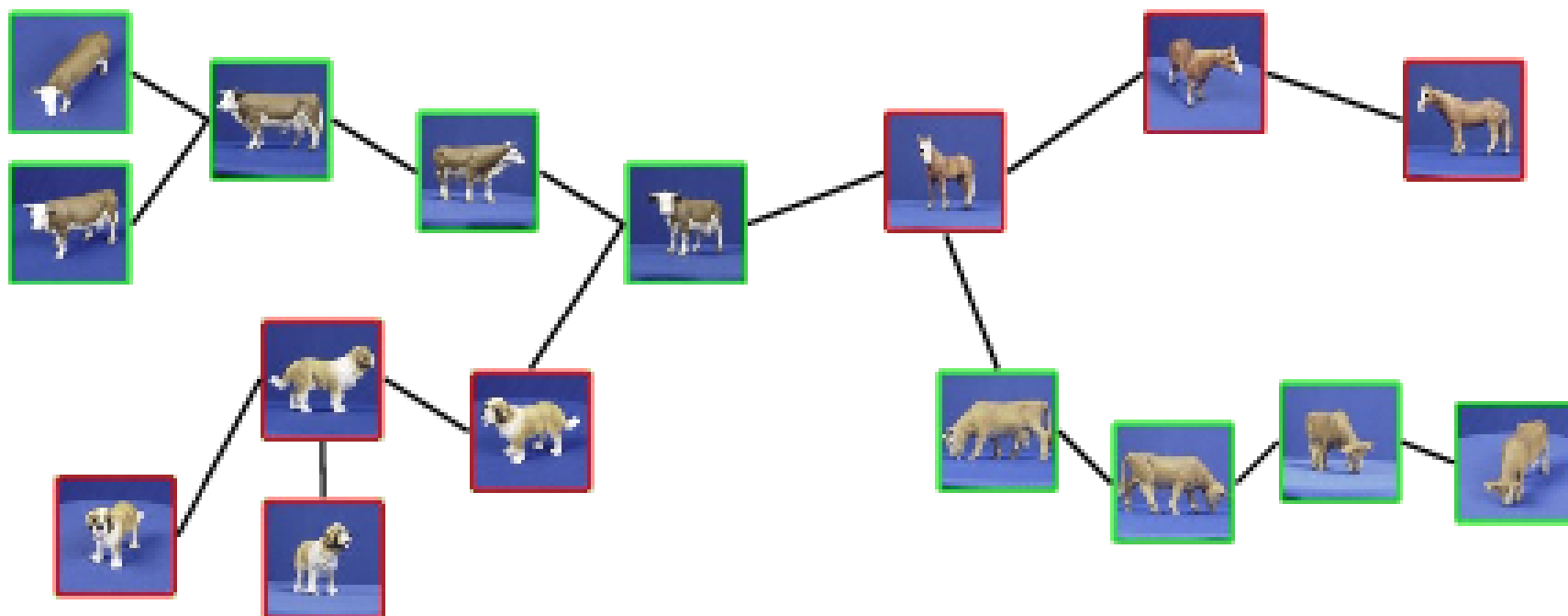
- Nesse caso, “x” recebe o mesmo rótulo de “a” com um custo de 0,2.
- Demais elementos da base de teste também são rotulados da mesma forma.



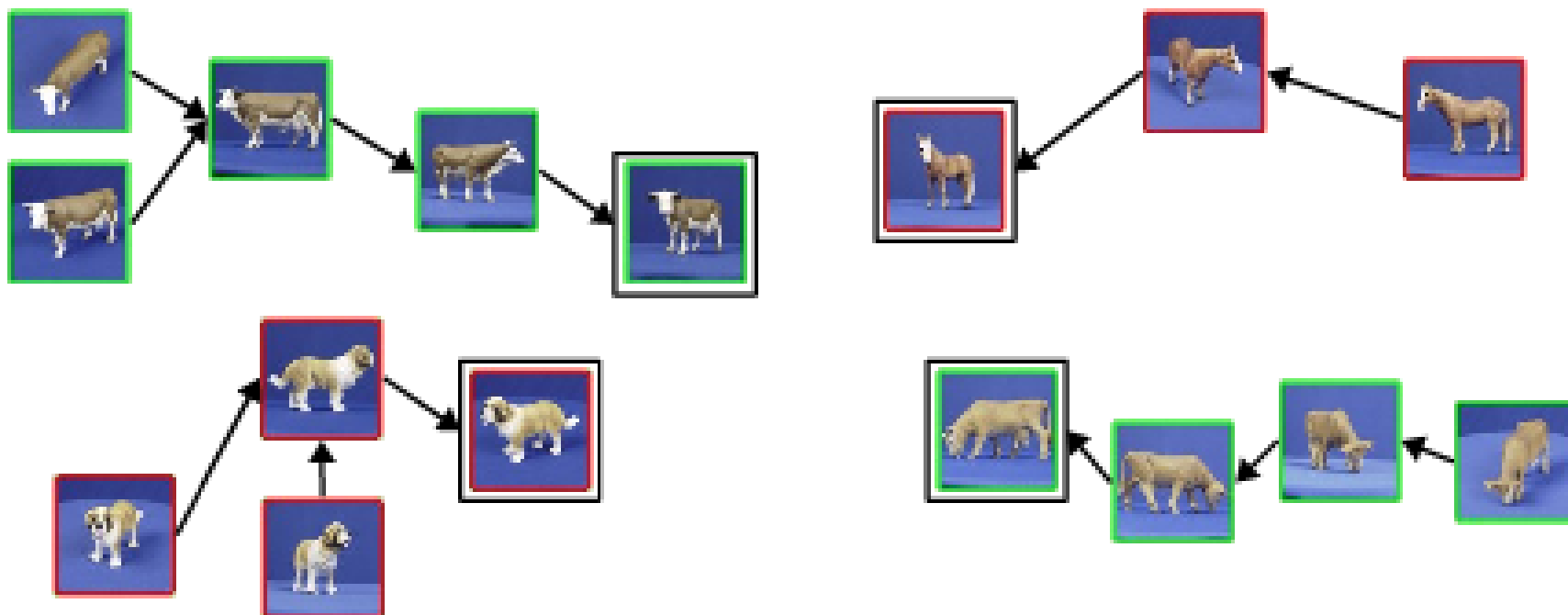
Exemplo



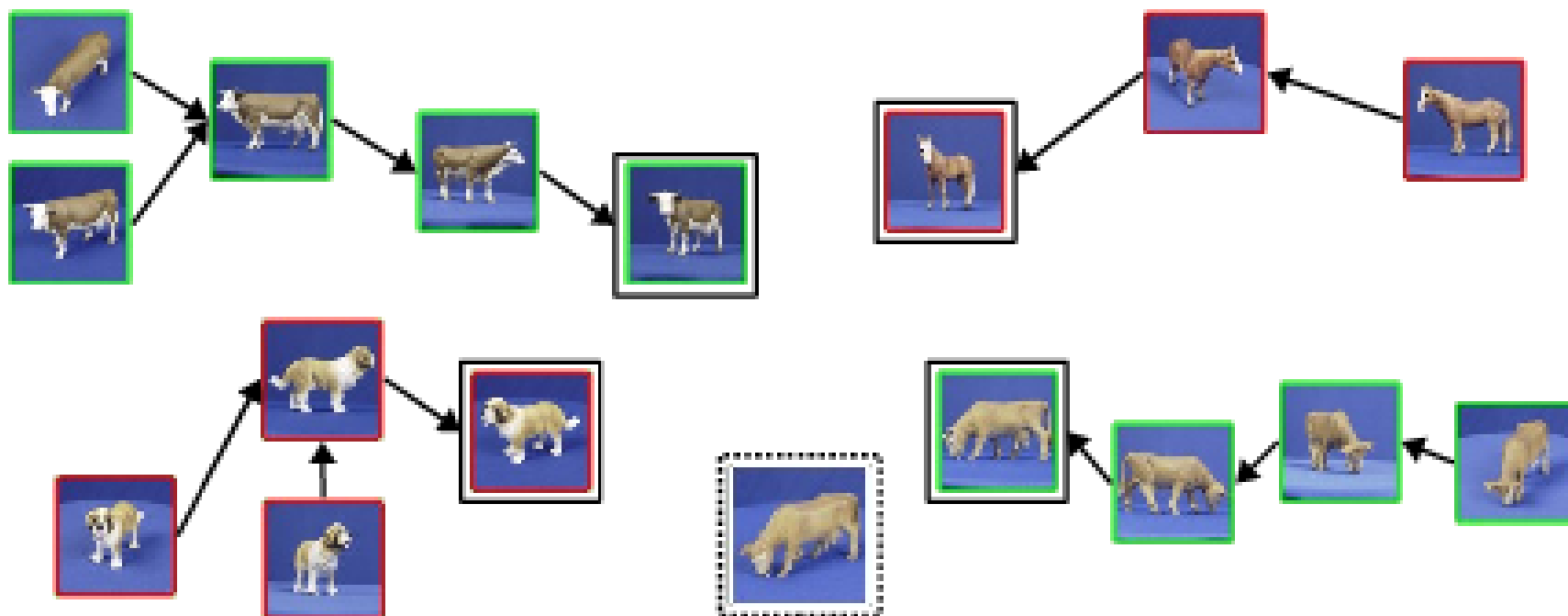
Exemplo



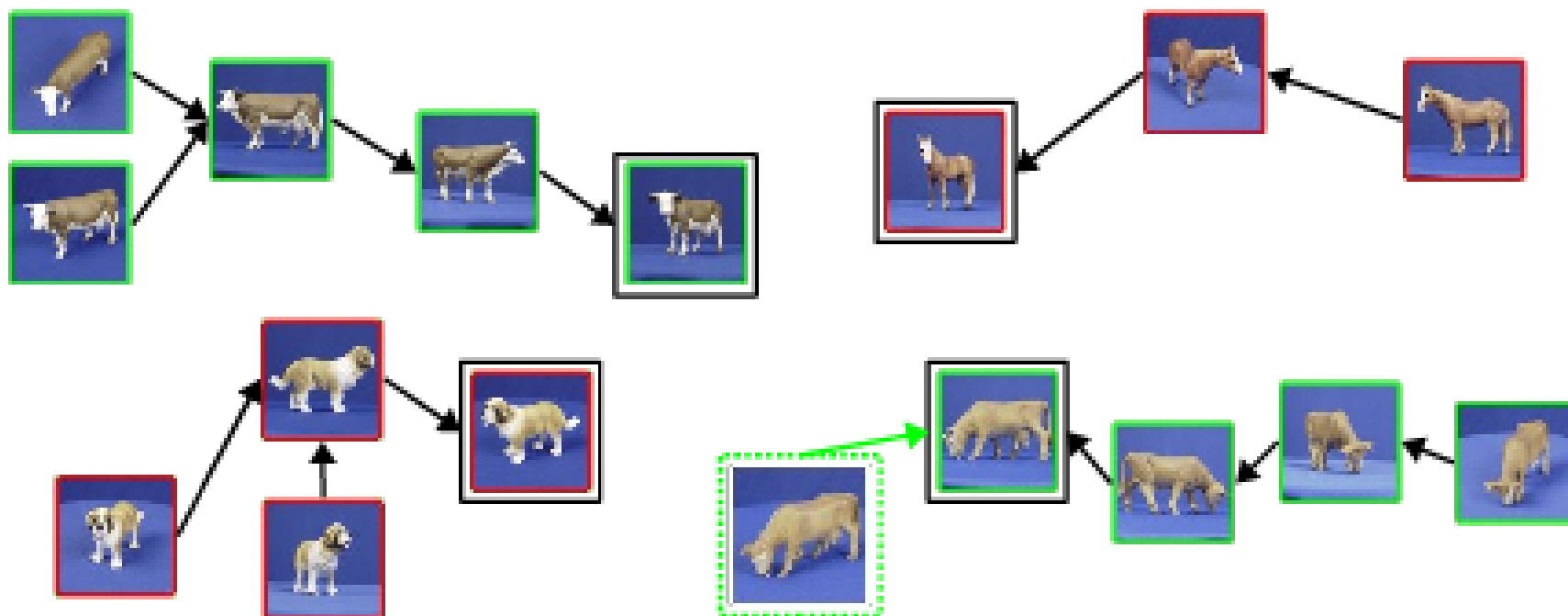
Exemplo



Exemplo



Exemplo



Referências

- Vídeo (aula):

<https://www.youtube.com/watch?v=IdLrgnb3HVc>

- Artigos:

- PAPA, J. P.; FALCAO, A. X.; MIRANDA, P. A. V.; SUZUKI, C. T. N.; MASCARENHAS, Nelson Delfino D' Ávila . Design of Robust Pattern Classifiers based on Optimum-Path Forests. In: 8th International Symposium on Mathematical Morphology, 2007, Rio de Janeiro. 8th International Symposium on Mathematical Morphology. São Jose dos Campos : MCT/INPE, 2007. p. 337-348.
- PAPA, J. P.; Falcao, A. X.; SUZUKI, C. T. N. Supervised pattern classification based on optimum-path forest. International Journal of Imaging Systems and Technology, v. 19, p. 120-131, 2009.
- PAPA, J. P.; FALCAO, A. X.; LEVADA, A. L. M.; CORREA, D.; Salvadeo, D.; MASCARENHAS, Nelson Delfino D' Ávila . Fast and Accurate Holistic Face Recognition using Optimum-Path Forest. In: 16th International Conference on Digital Signal Processing, 2009, Santorini, Grécia. Proceedings of the 16th International Conference on Digital Signal Processing, 2009. p. 1-6.
- PAPA, J. P.; FALCAO, A. X.; FREITAS, G. M.; AVILA, A. M. H. Robust Pruning of Training Patterns for Optimum-Path Forest Classification Applied to Satellite-Based Rainfall Occurrence Estimation. IEEE Geoscience and Remote Sensing Letters (Print), v. 7, p. 396-400, 2010.
- Iwashita, A. S.; Papa, João P.; Falcão, A. X.; Lotufo, R.A.; Oliveira, V.M.A.; de Albuquerque, Victor Hugo C.; Tavares, João Manuel R.S. Speeding Up Optimum-Path Forest Training by Path-cost Propagation. In: 21st International Conference on Pattern Recognition, 2012, Tsukuba. Proceedings of the 21st International Conference on Pattern Recognition, 2012. p. 1233-1236.